

N3-EXAM – 2026-10

SOLUCIÓN

Pregunta 1 (20 pts). Implemente en Python la función `buscar_llave_por_valor`, tal y como se describe en la documentación. No es necesario crear aserciones.

```
def buscar_llave_por_valor(diccionario: dict, valor_buscado: int) -> str:
    """
    Retorna solo la primera llave asociada al valor buscado en un diccionario. Se puede asumir que:
    - Las llaves del diccionario (si alguna) son siempre del tipo str.
    - Los valores del diccionario (si alguno) son siempre del tipo int.

    Args:
        diccionario (dict): Diccionario en el que se debe buscar el valor.
        buscado (int): Valor entero a buscar dentro del diccionario.

    Returns:
        str: Llave en la que se encuentra el valor buscado. Si no está, retorna 'NO_ENCONTRADO'.

    >>> buscar_llave_por_valor({}, 10) # Diccionario vacío
    'NO_ENCONTRADO'
    >>> buscar_llave_por_valor({"x": 1, "y": 2, "z": 3}, 5) # Valor no presente en el diccionario
    'NO_ENCONTRADO'
    >>> buscar_llave_por_valor({"a": 10, "b": 25, "c": 10, "d": 50}, 25) # Valor 25 asociado a "b"
    'b'
    >>> buscar_llave_por_valor({"a": 10, "b": 10, "c": 10, "d": 10}, 10) # Valor 10 repetido
    'a'
    >>> buscar_llave_por_valor({"a": -5, "c": 5, "z": 0}, 0) # Valor 0 asociado a "z"
    'z'
    """
    resultado = "NO_ENCONTRADO"

    llaves = list(diccionario.keys())

    indice = 0
    encontrado = False

    while indice < len(llaves) and not encontrado:
        llave_actual = llaves[indice]
        if diccionario[llave_actual] == valor_buscado:
            resultado = llave_actual
            encontrado = True
            indice += 1

    return resultado
```

Nota: Hay algoritmos en donde se puede usar `for` y `while` simultáneamente. Sin embargo, en el curso, los algoritmos son muy simples y tienden a solucionarse con un patrón de recorrido total o un patrón de recorrido parcial. Si se trata de este último, siempre se hará con `while` y a su gusto, con centinela o sin centinela, pero siempre `while` y con la respectiva variable de control.

Rúbrica de calificación de la Pregunta 1 (20 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-40): • 10 pts si cumple. • 5 pts si incumple una sola y la solución es correcta. • 0 pts si incumple 2 o más o la solución es errónea.			
La implementación de la función cumple con lo solicitado: • 10 pts si cumple a cabalidad con lo solicitado. • 5 pts si el algoritmo es parcialmente correcto, fallando en detalles muy menores. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, o no se presentó una solución.			
Total:			

Las siguientes preguntas se basan en el proyecto de Nivel 3: **CupiCharts**.

Utilice la siguiente información de contexto:

- El diccionario que representa a una canción, tiene las siguientes llaves:

Llave	Descripción	Tipo	Ejemplo
title	Título de la canción.	str	"Die With a Smile"
chart_week	Fecha en que alcanzó el chart en formato: "YYYY-MM-DD".	str	"2025-06-28"
performer	Artista o intérprete de la canción.	str	"Lady Gaga"
peak_pos	Posición máxima alcanzada en el chart.	int	1
wks_on_chart	Número de semanas que la canción ha estado en el chart.	int	44
album_name	Nombre del álbum al que pertenece la canción.	str	"Die With a Smile"
release_date	Fecha de lanzamiento de la canción en formato: "YYYY-MM-DD".	str	"2024-08-16"
popularity	Popularidad de la canción en una escala del 1 al 100.	int	98
explicit	Indica si la canción tiene contenido explícito.	bool	False
listeners	Número de oyentes que han escuchado la canción.	int	1412509
play_count	Número de reproducciones de la canción.	int	23765285
duration_s	Duración de la canción en segundos.	float	251.667

En el dataset (cupicharts.csv) también está el campo "genre", el cual es un string que representa el género musical al cual pertenece cada canción. Ejemplo: "pop".

Pregunta 2 (20 pts). Implemente en Python la función `calcular_total oyentes_por_artista`, tal y como se describe en la documentación (siguiente página). No debe crear *asepciones*.

Se le proporcionan los siguientes doctests de ejemplo para que comprenda aún mejor el propósito de la función:

```
>>> cupicharts = cargar_cupicharts("./cupicharts.csv") # Carga inicial

>>> calcular_total oyentes_por_artista(cupicharts, "Artista Inexistente") # Artista ausente
0

>>> calcular_total oyentes_por_artista({}, "Morgan Wallen") # Diccionario vacío
0

>>> cupicharts_nuevo = {
...     "pop": [
...         {"title": "Canción A", "performer": "Artista A", "listeners": 100},
...         {"title": "Canción B", "performer": "Artista B", "listeners": 200}
...     ],
...     "rock": [
...         {"title": "Canción C", "performer": "Artista A", "listeners": 300},
...         {"title": "Canción D", "performer": "Artista A", "listeners": 50}
...     ]
... } # Caso personalizado
>>> calcular_total oyentes_por_artista(cupicharts_nuevo, "Artista A") # Suma correcta
450

>>> calcular_total oyentes_por_artista(cupicharts_nuevo, "Artista B") # Única coincidencia
200
```

Debe responder en la siguiente página.

Rúbrica de calificación de la Pregunta 2 (20 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-40): • 10 pts si cumple. • 5 pts si incumple una sola y la solución es correcta. • 0 pts si incumple 2 o más o la solución es errónea.			
La implementación de la función cumple con lo solicitado: • 10 pts si cumple a cabalidad con lo solicitado. • 10 pts si el algoritmo es parcialmente correcto, fallando en detalles muy menores. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, o no se presentó una solución.			
Total:			

```

def calcular_total_oyentes_por_artista(cupicharts: dict, artista_buscado: str) -> int:
    """
    Calcula el total de oyentes de todas las canciones de un artista específico.

    Args:
        cupicharts (dict): Diccionario con la información de canciones.
        artista_buscado (str): Nombre del artista a buscar.

    Returns:
        int: Suma total del campo "listeners" de todas las canciones del artista.
        Si el artista no tiene canciones en el diccionario, retorna 0.
    """
    total_oyentes = 0

    for genero in cupicharts:
        lista_canciones = cupicharts[genero]
        for cancion in lista_canciones:
            if cancion["performer"] == artista_buscado:
                total_oyentes += cancion["listeners"]

    return total_oyentes

```

Pregunta 3 (30 pts). Implemente en Python la función `relacionar_album_con_canciones`, tal y como se describe en la documentación (siguiente página). No debe crear *doctests* ni aserciones.

Lista de chequeo:

- ☐ La función excluye correctamente a las canciones repetidas.

Se le proporciona el siguiente ejemplo de plan de solución:

- Inicializar el resultado por defecto.
- Recorrer todos los géneros del diccionario principal.
- Recorrer las canciones asociadas a cada género.
- Obtener, para cada canción, el nombre del álbum y el título.
- Verificar si el álbum ya existe como llave en el diccionario resultado.
 - Si no existe, crear una entrada para ese álbum con una lista vacía.
- Comprobar si el título de la canción ya está registrado en la lista de ese álbum.
 - Si no está repetido, agregarlo a la lista correspondiente.
- Al finalizar el recorrido, retornar el diccionario construido.

Debe responder en la siguiente página.

Rúbrica de calificación de la Pregunta 3 (30 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-40): • 10 pts si cumple. • 5 pts si incumple una sola y la solución es correcta. • 0 pts si incumple 2 o más o la solución es errónea.			
La implementación de la función cumple con lo solicitado: • 20 pts si cumple a cabalidad con lo solicitado. • 10 pts si el algoritmo es parcialmente correcto, falla en detalles muy menores o falla máximo un ítem de la lista de chequeo. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, falla dos o más ítems de la lista de chequeo o no se presentó una solución.			
Total:			

```
def relacionar_album_con_canciones(cupicharts: dict) -> dict:
    """
    Agrupa las canciones por álbum en un diccionario, donde cada llave es el nombre único de un
    álbum y su valor es la lista de canciones correspondientes, sin canciones duplicadas.

    Args:
        cupicharts (dict): Diccionario con la información de canciones.

    Returns:
        dict: Diccionario donde:
        - Cada llave (str) es el nombre de un álbum.
        - Cada valor (list) es una lista con los nombres de las canciones del álbum, sin duplicados.
        Si no hay álbumes o canciones, se retorna un diccionario vacío.
    """
    albumes = {}

    for genero in cupicharts:
        lista_canciones = cupicharts[genero]
        for cancion in lista_canciones:
            album = cancion["album_name"]
            if album not in albumes:
                albumes[album] = []

            if cancion["title"] not in albumes[album]:
                albumes[album].append(cancion["title"])

    return albumes
```

Pregunta 4 (30 pts). Implemente en Python la función `buscar_primera_cancion_entre_fechas`, tal y como se describe en la documentación (siguiente página). No debe crear `doctests` ni aserciones.

Se le proporciona el siguiente ejemplo plan de solución:

- Inicializar el resultado por defecto y variables necesarias para el recorrido.
- Recorrer la estructura principal y, dentro de ella, las canciones asociadas.
- En cada canción, verificar si la fecha de lanzamiento está dentro del rango dado.
- Cuando se encuentre una primera coincidencia, guardarla y concluir la búsqueda.
- Retornar la canción encontrada o el resultado por defecto si no hay coincidencias.

Debe responder en la siguiente página.

Rúbrica de calificación de la Pregunta 4 (30 pts):

Descripción	Cumple	Cumple parcialmente	Incumple
Cumple con las buenas prácticas del curso (BP-1 a BP-40): • 15 pts si cumple. • 7.5 pts si incumple una sola y la solución es correcta. • 0 pts si incumple 2 o más o la solución es errónea.			
La implementación de la función cumple con lo solicitado: • 15 pts si cumple a cabalidad con lo solicitado. • 7.5 pts si el algoritmo es parcialmente correcto, fallando en detalles muy menores. • 0 pts si el algoritmo es parcialmente correcto, pero contiene errores graves, o no se presentó una solución.			
Total:			

```

def buscar_primera_cancion_entre_fechas(cupicharts: dict, fecha_min: str, fecha_max: str) -> dict:
    """
    Busca la primera canción cuya fecha de lanzamiento esté entre dos fechas dadas. Son válidas las
    canciones cuya fecha de lanzamiento sea igual a fecha_min o fecha_max.

    Args:
        cupicharts (dict): Diccionario con la información de canciones.
        fecha_min (str): Fecha mínima de lanzamiento, en formato "YYYY-MM-DD".
        fecha_max (str): Fecha máxima de lanzamiento, en formato "YYYY-MM-DD".

    Returns:
        dict: Primer diccionario de canción que cumple la condición.
        Si no existe ninguna, retorna un diccionario vacío.
    """
    cancion_encontrada = {}
    generos = list(cupicharts.keys())

    i = 0
    encontrada = False

    while i < len(generos) and not encontrada:
        genero = generos[i]
        lista_canciones = cupicharts[genero]

        j = 0
        while j < len(lista_canciones) and not encontrada:
            cancion = lista_canciones[j]
            if fecha_min <= cancion["release_date"] <= fecha_max:
                cancion_encontrada = cancion
                encontrada = True
            j += 1

        i += 1

    return cancion_encontrada

```

Nota: Hay algoritmos en donde se puede usar for y while simultáneamente. Sin embargo, en el curso, los algoritmos son muy simples y tienden a solucionarse con un patrón de recorrido total o un patrón de recorrido parcial. Si se trata de este último, siempre se hará con while y a su gusto, con centinela o sin centinela, pero siempre while y con la respectiva variable de control.